

Kreacijski paterni – FitManager

1. Factory - Korišten

Factory Method definiše zajednički način kreiranja objekata, ali odluku o konkretnom tipu objekta prebacuje u posebnu fabričku klasu ili metodu. Na taj način ostatak sistema ne mora znati sve detalje inicijalizacije konkretnih klasa.

Korištenje: U sistemu FitManager postoje različite uloge korisnika: član, trener i administrator. Svi oni pripadaju istoj osnovnoj porodici korisnika, ali se u poslovnom smislu kreiraju sa različitim odgovornostima. Član dobija profil, članarine, rezervacije i QR kod, trener se povezuje sa grupnim treninzima, dok administrator generiše izvještaje i konfiguriše cijene članarina. Ako bi se ta logika ponavljala u kontrolerima ili servisima, registracija korisnika bi brzo postala nepregledna i teža za proširivanje.

Zato se uvodi `KorisnikFactory`, koja na osnovu vrijednosti `UlogaKorisnika` kreira odgovarajući tip korisnika: `Clan`, `Trener` ili `Administrator`. Ostatak sistema poziva fabriku i dobija validno kreiran objekat, bez direktnog poznavanja svih pravila inicijalizacije.

2. Builder - Korišten

Builder se koristi kada se složen objekat ne kreira odjednom, nego kroz više povezanih koraka. Svaki korak postavlja dio stanja objekta, a na kraju se dobija kompletan i konzistentan rezultat.

Korištenje: Klasa `PlanTreninga` u FitManager sistemu zavisi od više povezanih podataka: fitness cilja, BMI vrijednosti, BMI kategorije, intenziteta, sedmičnog plana i datuma kreiranja. Ovi podaci nisu potpuno nezavisni, jer se intenzitet i preporučeni plan mogu odrediti na osnovu cilja i BMI kategorije korisnika.

Zbog toga se uvodi `PlanTreningaBuilder`, koji postepeno sklapa plan treninga kroz metode kao što su `PostaviCilj`, `PostaviBmi`, `OdrediKategoriju`, `PostaviIntenzitet` i `GenerisiSedmicniPlan`. Nakon što su svi potrebni koraci završeni, metoda `Build` vraća potpuno formiran objekat `PlanTreninga`.

3. Singleton

Singleton osigurava da klasa ima samo jednu instancu u cijelom sistemu i da se toj instanci pristupa kroz jednu globalnu tačku.

U FitManager sistemu trenutno ne postoji domenska klasa kojoj je nužno ograničiti broj instanci na jednu. Entiteti kao što su `Korisnik`, `Clanarina`, `Rezervacija`, `GrupniTrening` i `PlanTreninga` moraju postojati u više primjeraka, jer predstavljaju

konkretne podatke iz baze. ASP.NET Core već upravlja životnim ciklusom infrastrukturnih servisa kao što su `ApplicationDbContext`, kontroleri i `Identity` komponente kroz dependency injection.

Zbog toga bi ručno dodavanje Singleton paterna u domenski model stvorilo nepotrebnu zavisnost i otežalo testiranje. Ako bi sistem kasnije imao centralni servis za audit log ili konfiguraciju, Singleton bi se mogao razmotriti, ali u trenutnom modelu nije potreban.

4. Abstract Factory

Abstract Factory omogućava kreiranje porodica povezanih objekata bez navođenja njihovih konkretnih klasa.

`FitManager` nema više paralelnih porodica proizvoda koje bi se morale kreirati zajedno. Na primjer, sistem ne kreira različite porodice objekata za posebne platforme, teme aplikacije ili nezavisne module sa istim interfejsima. Korisnici, članarine, treninzi i rezervacije predstavljaju jednu domensku cjelinu, a ne više zamjenjivih porodica objekata.

Zbog toga bi Abstract Factory u ovom slučaju dodao previše apstrakcije. Jednostavnije i jasnije rješenje je koristiti `KorisnikFactory` samo tamo gdje stvarno postoji potreba za centralizovanim kreiranjem srodnih tipova korisnika.

5. Prototype

Prototype omogućava kreiranje novih objekata kopiranjem postojećih instanci, umjesto kreiranjem objekta kroz konstruktor ili fabriku.

U `FitManager` sistemu većina objekata nastaje na osnovu novih korisničkih unosa ili rezultata poslovne logike. Rezervacija se veže za tačno određenog člana i grupni trening, članarina ima konkretan period važenja, a plan treninga se formira prema trenutnim podacima člana. Ti objekti imaju vlastiti identitet u bazi podataka i ne bi ih bilo ispravno praviti kao kopije postojećih objekata.

Prototype bi imao smisla samo ako bi sistem uveo šablone treninga koji se često kopiraju i zatim prilagođavaju pojedinačnim članovima. Pošto trenutni model već koristi `PlanTreningaBuilder` za formiranje individualnog plana, Prototype nije potreban.